Checklist genérico para crear juegos Android

Flutter + Flame como stack principal · base para la skill «creación de juegos Android»

13 fases | Prioridades: crítico · importante · deseable | agnóstico / flutter-flame | ⚠ revisar vigencia

Preparado para Juan Manuel de Villeros

19 de octubre de 2026 (hora de Bogotá, UTC-5)

Destilado de tres proyectos propios y de las guías oficiales de Android, Google Play, Flutter y Flame

Cómo usar este checklist

- Cada ítem lleva casilla, acción en imperativo, **Por qué**, **Verificar** (comando, archivo o lugar en Play Console), **Prioridad** y **Fuente** oficial cuando la hay.
- **Prioridad:** `crítico` bloquea la publicación o rompe la experiencia en dispositivos actuales; `importante` lo exigen o recomiendan con fuerza las guías de calidad; `deseable` es una mejora.
- **Alcance:** `agnóstico` aplica a cualquier stack (Unity, Godot, Kotlin nativo…); `flutter/flame` es específico del stack principal.
- ⚠ **Revisar vigencia** marca requisitos con fecha que cambian año a año (API objetivo, 16 KB, políticas). Hay que comprobarlos en la fuente antes de cada lanzamiento.
- Los nombres de archivo siguen la plantilla de `flutter create` (`android/app/...`, `lib/...`).

Resumen por fase (prioridad principal de cada ítem)

Fase	Ítems	Crítico	Importante	Deseable
Fase 0 · Inicio, plantilla y metodología	9	1	6	2
Fase 1 · Configuración del proyecto y build	10	5	4	1
Fase 2 · Pantalla, escala, orientación, edge-to-edge y pantallas grandes	9	2	6	1
Fase 3 · Ciclo de vida, estado y guardado	7	1	4	2
Fase 4 · Audio	6	1	3	2
Fase 5 · Entrada y retroceso predictivo	6	0	4	2
Fase 6 · Rendimiento	6	0	4	2
Fase 7 · Accesibilidad e internacionalización	6	0	3	3
Fase 8 · Errores, crashes y símbolos	5	0	5	0
Fase 9 · Pruebas y CI	8	0	6	2
Fase 10 · Firma y release	6	5	0	1
Fase 11 · Publicación en Google Play y políticas	10	6	3	1
Fase 12 · Mantenimiento post-lanzamiento	6	1	3	2
Total	**94**	**22**	**51**	**21**

Fase 0 Inicio, plantilla y metodología

- [ ] **Escribir la intención y la especificación antes del código** (intención → spec → plan → decisiones). agnóstico
  - Por qué: un agente o un desarrollador sin spec produce retrabajo; la spec es el contrato de "hecho".
  - Verificar: existen `docs/00_intent.md`, `docs/00_spec.md`, `docs/01_plan.md` y `docs/02_decisions.md` (ADR).
  - Prioridad: importante.

- [ ] **Mantener un archivo de estado reanudable** (fase actual, último checkpoint verde, bloqueos, siguiente paso). agnóstico
  - Por qué: permite retomar sesiones (humanas o de agentes) sin perder contexto.
  - Verificar: `docs/03_state.md` actualizado al cerrar cada fase.
  - Prioridad: importante.

- [ ] **Definir reglas para agentes y de parada**: Play, pagos, secretos o keystore, merge a `main` y release público los decide siempre el dueño. agnóstico
  - Por qué: evita acciones irreversibles hechas por automatización.
  - Verificar: `AGENTS.md` con prohibiciones, nivel de autonomía y presupuestos.
  - Prioridad: importante.

- [ ] **Crear un arnés único de verificación** (`verify.sh`) que usen igual el agente y el CI. agnóstico
  - Por qué: una sola definición de "verde" evita que un check se quede fuera.
  - Verificar: `./verify.sh` termina con código 0; `./verify.sh --no-apk` sirve para iterar rápido.
  - Prioridad: importante.

- [ ] **Escribir un guion de aceptación humana** que contenga solo lo que una máquina no puede juzgar (sensación, curva de dificultad, legibilidad). agnóstico
  - Por qué: separa la validación automática de la de gusto y hace repetible la demo.
  - Verificar: `docs/04_acceptance.md` con casillas y veredicto.
  - Prioridad: deseable.

- [ ] **Fijar el `applicationId` definitivo antes de la primera subida a Play** (dominio inverso propio). agnóstico
  - Por qué: no se puede cambiar una vez subido.
  - Verificar: `applicationId` y `namespace` en `android/app/build.gradle.kts`, más el paquete de `MainActivity.kt`.
  - Prioridad: crítico.
  - Fuente: https://docs.flutter.dev/deployment/android

- [ ] **Comprobar que el nombre del juego está disponible** (Play, dominio, marcas). agnóstico
  - Por qué: renombrar después cuesta ficha, icono, dominio y política.
  - Verificar: búsqueda en Play y web; dominio para la política de privacidad.
  - Prioridad: deseable.

- [ ] **Centralizar constantes de juego y textos** (`game_config.dart`, `app_strings.dart`). flutter/flame
  - Por qué: facilita el ajuste de balance, los tests y la traducción posterior.
  - Verificar: `rg "fontSize:|Duration\(" lib` no muestra números mágicos dispersos en la lógica.
  - Prioridad: importante.

- [ ] **Separar la lógica pura (sin Flame ni Flutter) de los componentes**. flutter/flame
  - Por qué: la lógica pura se prueba con tests rápidos y sin plataforma.
  - Verificar: `lib/logic` o `lib/managers` no importan `package:flame` y tienen tests.
  - Prioridad: **importante**.

Fase 1 Configuración del proyecto y build

- [ ] **Usar los SDK por defecto de Flutter, o fijarlos a conciencia** (`compileSdk` , `targetSdk` , `minSdk`).
  flutter/flame ⚠ revisar vigencia
  - Por qué: `flutter.targetSdkVersion` sigue el requisito de Play al actualizar Flutter. Si se fija un número, hay que revisarlo cada año.
  - Verificar: `android/app/build.gradle.kts` ; valor efectivo en `build/app/intermediates/merged_manifests/release/.../AndroidManifest.xml` (`<uses-sdk>`).
  - Prioridad: crítico .
  - Fuente: https://docs.flutter.dev/deployment/android

- [ ] **Apuntar al API exigido por Play**: desde el **31-ago-2026, API 36** para apps nuevas y actualizaciones; las existentes necesitan **API 35 o más** para seguir visibles. agnóstico ⚠ revisar vigencia
  - Por qué: Play rechaza la subida si no se cumple.
  - Verificar: manifiesto fusionado (`targetSdkVersion="36"`); Play Console › Explorador de app bundles.
  - Prioridad: crítico .
  - Fuente: https://support.google.com/googleplay/android-developer/answer/11926878 · https://developer.android.com/google/play/requirements/target-sdk

- [ ] **Usar un toolchain compatible con 16 KB** (AGP 8.5.1 o más, NDK r28 o más, Flutter 3.35 o más) y actualizar los plugins con código nativo. agnóstico ⚠ revisar vigencia
  - Por qué: Play exige 16 KB para apps con target 35+; las actualizaciones que no cumplan se bloquean desde el 1-feb-2027.
  - Verificar: `unzip` del AAB y `readelf -lW base/lib/arm64-v8a/*.so | grep LOAD` (alineación de 0x4000 o más); Play Console › Explorador de app bundles.
  - Prioridad: crítico .
  - Fuente: https://developer.android.com/guide/practices/page-sizes

- [ ] **Versionar desde `pubspec.yaml`** (`version: X.Y.Z+N`), subir `N` (versionCode) en cada AAB y mantener una sola fuente de versión en la documentación. flutter/flame
  - Por qué: Play rechaza un versionCode repetido, y las versiones contradictorias en la documentación confunden.
  - Verificar: `grep ^version pubspec.yaml` ; que el README y el plan no citen otra versión.
  - Prioridad: crítico .

- [ ] **Confiar en R8 (activo por defecto en release) y añadir `proguard-rules.pro` solo si un plugin lo pide**.
  flutter/flame
  - Por qué: reduce el tamaño; las reglas innecesarias esconden problemas.
  - Verificar: `flutter build appbundle` sin errores de R8; la app arranca en release.
  - Prioridad: importante .
  - Fuente: https://docs.flutter.dev/deployment/android

- [ ] **Declarar solo los permisos que el código usa de verdad**; no añadir `INTERNET` si el juego es offline.
  agnóstico
  - Por qué: Minimize_Permissions de la guía de calidad; además, la política de privacidad y Data safety deben coincidir.
  - Verificar: `grep uses-permission` en el manifiesto fusionado; buscar en `lib/` el uso real de cada permiso (por ejemplo, `WAKE_LOCK` sin plugin wakelock sobra).
  - Prioridad: importante .
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Eliminar las dependencias sin uso y empaquetar las fuentes en lugar de descargarlas** (con su licencia, por ejemplo OFL). flutter/flame
  - Por qué: una fuente descargada en tiempo de ejecución necesita red y cambia la superficie de privacidad.
  - Verificar: `rg "import 'package:<dep>" lib` para cada dependencia; `flutter pub deps`; fuentes en `assets/fonts/`.
  - Prioridad: **deseable**.

- [ ] **Declarar todos los assets en `pubspec.yaml`** y comprobar que entran en el build. flutter/flame
  - Por qué: un asset no declarado falla solo en tiempo de ejecución.
  - Verificar: `unzip -l build/app/outputs/flutter-apk/app-release.apk | grep flutter_assets`.
  - Prioridad: **importante**.

- [ ] **Crear un icono adaptativo propio**: legacy, `mipmap-anydpi-v26/ic_launcher.xml` con foreground y background, y capa `<monochrome>`. agnóstico
  - Por qué: el icono por defecto de la plantilla no es publicable, y Android 13+ usa el monocromo para los iconos temáticos.
  - Verificar: `md5sum` de `mipmap-*/ic_launcher.png` distinto al de la plantilla de Flutter; existe el XML adaptativo.
  - Prioridad: **crítico** (icono propio) / **deseable** (monocromo).
  - Fuente: https://docs.flutter.dev/deployment/android

- [ ] **Dar a `LaunchTheme` y `NormalTheme` el color de fondo del juego** (no `Theme.Light` si el juego es oscuro). flutter/flame
  - Por qué: evita el destello blanco antes del primer frame.
  - Verificar: `values/styles.xml`, `values-night/styles.xml` y `drawable*/launch_background.xml` con `@color/splash_background`; arranque en frío en modo claro.
  - Prioridad: **importante**.

```
<!-- values/colors.xml -->
<color name="splash_background">#FF0A0A14</color>
<!-- drawable/launch_background.xml -->
<layer-list xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:drawable="@color/splash_background" />
</layer-list>
```

Fase 2 Pantalla, escala, orientación, edge-to-edge y pantallas grandes

- [ ] **Declarar la app como juego**: `android:appCategory="game"` en `<application>`. agnóstico ⚠ revisar vigencia
  - Por qué: con target 36, Android 16 ignora `screenOrientation`, `resizeableActivity` y los límites de proporción en pantallas de 600 dp o más, **excepto en juegos**.
  - Verificar: manifiesto fusionado; emulador de tablet o plegable con API 36 que respete la orientación.
  - Prioridad: crítico.
  - Fuente: https://developer.android.com/about/versions/16/behavior-changes-16 · https://developer.android.com/develop/adaptive-apps/guides/app-orientation-aspect-ratio-resizability

```
<application
    android:label="MiJuego"
    android:appCategory="game"
    android:enableOnBackInvokedCallback="true"
    ...>
```

- [ ] **Fijar la orientación en dos sitios**: manifiesto (`portrait` o `sensorLandscape`) y `SystemChrome.setPreferredOrientations`. flutter/flame
  - Por qué: el manifiesto evita el giro inicial y Dart lo asegura en tiempo de ejecución.
  - Verificar: girar el dispositivo en cada pantalla.
  - Prioridad: importante.

- [ ] **Diseñar para edge-to-edge**: con target 35/36 es obligatorio y en Flutter con target 36 los modos distintos de `edgeToEdge` no tienen efecto. Usar `SystemUiMode.edgeToEdge` con barras transparentes y no usar `windowOptOutEdgeToEdgeEnforcement`. flutter/flame ⚠ revisar vigencia
  - Por qué: si se asume `immersiveSticky`, el HUD puede quedar bajo las barras o el notch.
  - Verificar: `rg "setEnabledSystemUIMode|windowOptOutEdgeToEdge" lib android`; prueba en un Android 16 físico con navegación por gestos y por botones.
  - Prioridad: crítico.
  - Fuente: https://api.flutter.dev/flutter/services/SystemChrome/setEnabledSystemUIMode.html · https://docs.flutter.dev/release/breaking-changes/default-systemuimode-edge-to-edge

```
SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
SystemChrome.setSystemUIOverlayStyle(const SystemUiOverlayStyle(
  statusBarColor: Colors.transparent,
  systemNavigationBarColor: Colors.transparent,
));
```

- [ ] **Respetar los insets** (barras, notch, recortes, zonas de gesto) en HUD, menús y controles táctiles: `SafeArea` o `MediaQuery.paddingOf`, con márgenes simétricos en horizontal. flutter/flame
  - Por qué: un joystick o un botón bajo el notch o la zona de gesto deja de funcionar.
  - Verificar: emulador con recorte (Settings › Developer options › Display cutout); test de widget con `padding` simulado.
  - Prioridad: importante.

- [ ] **Separar el área jugable del HUD por construcción**: ningún panel que absorba toques encima de la zona de juego. agnóstico
  - Por qué: elementos jugables bajo el HUD no se pueden tocar; es un fallo típico de los mockups "a sangre".
  - Verificar: test de widget en 3 o 4 tamaños (por ejemplo 360×640, 412×915, 800×360, tablet): intersección área ∩ HUD = ∅ y el hit-test llega al `Listener` del juego.
  - Prioridad: importante.

- [ ] **Escalar el mundo de juego de forma proporcional** (cámara o `FixedResolutionViewport`, `FittedBox` en tableros fijos, escala por lado corto) y probar varias proporciones (16:9 a 21:9, tablet). flutter/flame
  - Por qué: Fullscreen_Display (sin letterbox) y la paridad entre dispositivos.
  - Verificar: tests de layout en varios `Size`; capturas E2E.
  - Prioridad: **importante**.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Decidir de forma explícita la política de tamaño de letra**: por defecto respetarlo; si el HUD no lo admite, acotarlo en vez de anularlo en toda la app. flutter/flame
  - Por qué: la accesibilidad pide admitir letra grande; anularla en menús y ajustes perjudica a quien la necesita.
  - Verificar: test con `TextScaler.linear(1.3)` y `2.0` sin overflow; `rg "textScaler|withNoTextScaling" lib`.
  - Prioridad: **importante**.
  - Fuente: https://docs.flutter.dev/ui/accessibility/ui-design-and-styling

```
MaterialApp(
  builder: (context, child) {
    final mq = MediaQuery.of(context);
    return MediaQuery(
      data: mq.copyWith(
        textScaler: mq.textScaler.clamp(minScaleFactor: 1.0, maxScaleFactor: 1.3),
      ),
      child: child!,
    );
  },
);
// Para el HUD de partida (si es imprescindible): MediaQuery.withNoTextScaling(child: hud)
```

- [ ] **Probar el ajuste «Tamaño de pantalla» del sistema** (cambia la densidad, no la letra); si rompe proporciones, normalizar con la densidad estable (`DisplayMetrics.DENSITY_DEVICE_STABLE`). agnóstico
  - Por qué: con Display size ampliado, los controles de tamaño lógico fijo dejan de cuadrar con el mundo.
  - Verificar: Settings › Display › Display size al mínimo y al máximo; test de proporción.
  - Prioridad: **deseable**.

- [ ] **Probar en pantallas grandes, plegables y multiventana** (emulador de plegable de 7,6", tablet tipo Pixel C, ventana de escritorio). agnóstico
  - Por qué: la guía de calidad pide estos emuladores como mínimo; además, calidad adaptativa mejora el ranking en Play.
  - Verificar: AVD Manager; Play Console › Calidad › pantallas grandes.
  - Prioridad: **importante**.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

Fase 3 Ciclo de vida, estado y guardado

- [ ] **Pausar el bucle de juego en segundo plano**: Flame lo hace por defecto (`pauseWhenBackgrounded = true`); si se sobrescribe `lifecycleStateChange`, llamar siempre a `super`. flutter/flame
  - Por qué: evita saltos de `dt` enormes al volver y el consumo de batería (Sleep_Resume, Lock_Resume).
  - Verificar: bloquear y desbloquear, ir a Recents y volver; `rg "lifecycleStateChange|pauseWhenBackgrounded" lib`.
  - Prioridad: crítico.
  - Fuente: https://docs.flame-engine.org/latest/flame/game.html

```
@override
void lifecycleStateChange(AppLifecycleState state) {
  super.lifecycleStateChange(state); // mantiene la pausa automática de Flame
  switch (state) {
    case AppLifecycleState.paused:
    case AppLifecycleState.hidden:
    case AppLifecycleState.detached:
      audio.pauseAll();
    case AppLifecycleState.resumed:
      if (session.isPlaying) { session.pause(); overlays.add('pause'); }
    case AppLifecycleState.inactive:
      break;
  }
}
```

- [ ] **Al volver del segundo plano, mostrar un menú de pausa en lugar de reanudar en caliente.** agnóstico
  - Por qué: el jugador no pierde la partida por una notificación o una llamada.
  - Verificar: en partida, bajar la cortina de notificaciones e ir a Home; al volver aparece la pausa.
  - Prioridad: importante.

- [ ] **Observar el ciclo de vida también desde la pantalla de partida** (`WidgetsBindingObserver`) para el audio y los timers, y adoptar el estado actual al abrir la pantalla. flutter/flame
  - Por qué: una retención de audio "pegada" de una partida anterior deja el juego mudo; los `Timer.periodic` siguen corriendo en segundo plano.
  - Verificar: test de widget con `tester.binding.handleAppLifecycleStateChanged(...)`.
  - Prioridad: importante.

```
class _GameScreenState extends State<GameScreen> with WidgetsBindingObserver {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    _sync(WidgetsBinding.instance.lifecycleState); // adoptar el estado actual
  }
  @override
  void didChangeAppLifecycleState(AppLifecycleState s) => _sync(s);
  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    _ticker?.cancel();
    super.dispose();
  }
}
```

- [ ] **Guardar progreso y estadísticas en momentos seguros** (fin de nivel, pausa, salida) con escritura fail-soft. agnóstico
  - Por qué: State_Preservation; una excepción de almacenamiento no debe tumbar la partida.
  - Verificar: cerrar la app desde Recents a mitad de nivel y reabrir; test de la capa de persistencia con `SharedPreferences.setMockInitialValues`.
  - Prioridad: **importante**.

- [ ] **Decidir qué pasa con la partida en curso si el sistema mata el proceso** (restaurar o volver al menú de forma explícita). agnóstico
  - Por qué: la guía de calidad pide restaurar el estado si pasaron pocos minutos.
  - Verificar: `adb shell am kill <paquete>` con la app en segundo plano y volver.
  - Prioridad: **deseable**.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Ofrecer «Borrar datos o estadísticas» con confirmación**. agnóstico
  - Por qué: control del usuario; la política de privacidad debe explicar cómo borrar los datos.
  - Verificar: flujo en Ajustes; test de reset.
  - Prioridad: **deseable**.

- [ ] **Configurar el Auto Backup** (`android:dataExtractionRules` y `android:fullBackupContent`) para incluir las preferencias. agnóstico
  - Por qué: App_Data_Backup; el progreso sobrevive a un cambio de móvil.
  - Verificar: `adb shell bmgr backupnow <paquete>`, desinstalar, reinstalar y comprobar.
  - Prioridad: **importante**.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

Fase 4 Audio

- [ ] **Hacer el audio fail-soft**: si falla el init o un asset, el juego sigue y solo enmudece esa voz. agnóstico
  - Por qué: los fallos de códec o de dispositivo no deben provocar un crash.
  - Verificar: test con un asset inexistente; `try/catch` en `AudioManager`.
  - Prioridad: importante.

- [ ] **Pedir y ceder el foco de audio y reaccionar a llamadas u otras apps**. agnóstico
  - Por qué: Audio_Focus_Request y Audio_Focus_Change.
  - Verificar: reproducir música en otra app o recibir una llamada durante la partida.
  - Prioridad: importante.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

```
// audioplayers
await AudioPlayer.global.setAudioContext(AudioContext(
  android: const AudioContextAndroid(
    usageType: AndroidUsageType.game,
    contentType: AndroidContentType.music,
    audioFocus: AndroidAudioFocus.gain,
  ),
));
```

- [ ] **Silenciar todo en segundo plano y no usar un servicio en primer plano** (los juegos no reproducen audio en segundo plano). agnóstico
  - Por qué: Background_Service_Optimization; nadie quiere oír el juego con la pantalla bloqueada.
  - Verificar: ir a Home y bloquear el dispositivo; silencio inmediato.
  - Prioridad: crítico.

- [ ] **Liberar los reproductores de forma ordenada** (`await pause()` y luego `dispose()`; no hacer `stop` y `dispose` sin esperar). flutter/flame
  - Por qué: evita las condiciones de carrera y los crashes nativos al salir de la partida.
  - Verificar: entrar y salir de la partida 20 veces seguidas; logcat sin excepciones.
  - Prioridad: importante.

- [ ] **Añadir ajustes de sonido** (master, música y SFX por separado, mute) y persistirlos. agnóstico
  - Por qué: es una expectativa básica del jugador; muchos juegan sin sonido.
  - Verificar: test de la capa de ajustes; reiniciar la app conserva la mezcla.
  - Prioridad: deseable.

- [ ] **Precargar los SFX y los loops** y elegir formatos adecuados (OGG o MP3 cortos; los procedurales si encajan). agnóstico
  - Por qué: Audio_Playback_Start (menos de 1 s) y nada de tirones al primer disparo.
  - Verificar: perfil del primer evento de sonido.
  - Prioridad: deseable.

Fase 5 Entrada y retroceso predictivo

- [ ] **Activar el retroceso predictivo**: `android:enableOnBackInvokedCallback="true"` en `<application>`; con target 36 la animación es por defecto y ya no se llama a `onBackPressed`. agnóstico ⚠ revisar vigencia
  - Por qué: la interceptación heredada de BACK deja de funcionar en Android 16.
  - Verificar: manifiesto fusionado; gesto atrás en Android 14+ (aparece la vista previa al menú o a Home).
  - Prioridad: importante.
  - Fuente: https://docs.flutter.dev/platform-integration/android/predictive-back · https://developer.android.com/guide/navigation/custom-back/predictive-back-gesture

- [ ] **Proteger la partida en curso con `PopScope`** (`canPop` declarativo): un gesto atrás pausa y pide confirmación, y guarda el intento antes de salir. flutter/flame
  - Por qué: State_Preservation pide evitar la pérdida accidental por navegación atrás; los gestos laterales chocan con los joysticks.
  - Verificar: gesto atrás en plena partida, con navegación por gestos y por botones; test de widget con `tester.binding.handlePopRoute()`.
  - Prioridad: importante.

```
PopScope(
  canPop: false,
  onPopInvokedWithResult: (didPop, _) async {
    if (didPop) return;
    game.pause();
    final leave = await showExitDialog(context);
    if (leave && context.mounted) {
      await saveAttempt();
      Navigator.of(context).pop();
    }
  },
  child: GameWidget(game: game),
)
```

- [ ] **Seguir la navegación estándar**: atrás vuelve a la pantalla anterior y desde el menú principal sale; nada de botones "atrás" propios como única vía. agnóstico
  - Por qué: Back_Button_Nav y Back_Gesture_Nav.
  - Verificar: recorrer todas las pantallas pulsando atrás.
  - Prioridad: importante.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Hacer el multitáctil robusto**: rastrear los punteros por id, reenviar siempre `move` y `up`, y no dejar punteros "colgados" en pellizcos o con dos joysticks. flutter/flame
  - Por qué: un `up` perdido deja un control fijo o un pellizco fantasma.
  - Verificar: tests de gestos con varios punteros y órdenes de liberación distintos; pruebas manuales soltando en orden inverso.
  - Prioridad: importante.

- [ ] **Mantener los controles táctiles fuera de las zonas de gesto del sistema** (bordes) o configurables (posición y lado intercambiable). agnóstico
  - Por qué: los bordes disparan el gesto atrás o Home.
  - Verificar: `MediaQuery.systemGestureInsetsOf`; personalizador de controles.
  - Prioridad: deseable.

- [ ] **Ofrecer háptica opcional** detrás de una interfaz inyectable (para hacer tests con un fake). `flutter/flame`
  - Por qué: mejora la sensación; hay jugadores que la desactivan.
  - Verificar: test con un fake de `Haptics`; ajuste on/off.
  - Prioridad: **deseable**.

Fase 6 Rendimiento

- [ ] **Fijar como objetivo 60 FPS en gama media** (16,7 ms por frame) y medirlo en un dispositivo real en modo `--profile`. agnóstico
  - Por qué: Rendering_Performance.
  - Verificar: `flutter run --profile` con DevTools › Performance; un benchmark propio (`--dart-define=BENCH=true`) que imprima p50 y p95.
  - Prioridad: **importante** .
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **No asignar memoria en `update()` ni en `render()`** : cachear `Paint` , `Path` , `TextPainter` y shaders, y reutilizar segmentos o partículas con pools. flutter/flame
  - Por qué: la presión de GC provoca tirones.
  - Verificar: revisión de código; DevTools › Memory sin pico de GC en partida.
  - Prioridad: **importante** .

- [ ] **Limitar el coste de los cálculos pesados** (similitud, IA, generación): calcularlos a una frecuencia acotada, fuera del frame o en un `Isolate` . agnóstico
  - Por qué: evita ANR y frames largos.
  - Verificar: benchmark de la función; tiempos en el log.
  - Prioridad: **importante** .

- [ ] **Mostrar feedback si el arranque pasa de 2 s** y precargar los assets fuera de la partida. agnóstico
  - Por qué: App_Startup_Time.
  - Verificar: arranque en frío cronometrado; Play Console › Android vitals › Inicio.
  - Prioridad: **importante** .
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Vigilar el tamaño del AAB** (comprimir los vídeos y las imágenes grandes; si crece mucho, usar Play Asset Delivery). agnóstico
  - Por qué: el tamaño de descarga afecta a la conversión y tiene límites por módulo en Play.
  - Verificar: `ls -la build/app/outputs/bundle/release/` ; Play Console › Explorador de app bundles › tamaño de descarga.
  - Prioridad: **deseable** .

- [ ] **Comprobar la temperatura y el consumo de batería** en sesiones de 5 a 10 minutos. agnóstico
  - Por qué: Power_Management; los juegos largos calientan el dispositivo.
  - Verificar: prueba manual; Android Studio › Power Profiler.
  - Prioridad: **deseable** .

Fase 7 Accesibilidad e internacionalización

- [ ] **Etiquetar los controles con `Semantics` o `semanticsLabel`** (botones de menú, ajustes, iconos). flutter/flame
  - Por qué: Content_Description; TalkBack lee los menús.
  - Verificar: TalkBack activado; `rg "Semantics|semanticsLabel" lib`; `meetsGuideline(labeledTapTargetGuideline)` en tests.
  - Prioridad: **importante**.
  - Fuente: https://docs.flutter.dev/ui/accessibility-and-internationalization/accessibility

- [ ] **Usar objetivos táctiles de 48 dp o más y contraste de 4,5:1 o más** (3:1 en texto grande). agnóstico
  - Por qué: Touch_Target_Size y Visual_Contrast.
  - Verificar: Accessibility Scanner; `meetsGuideline(androidTapTargetGuideline)` y `textContrastGuideline`.
  - Prioridad: **importante**.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

```
testWidgets('a11y', (tester) async {
  final handle = tester.ensureSemantics();
  await tester.pumpWidget(const MyApp());
  await expectLater(tester, meetsGuideline(androidTapTargetGuideline));
  await expectLater(tester, meetsGuideline(labeledTapTargetGuideline));
  await expectLater(tester, meetsGuideline(textContrastGuideline));
  handle.dispose();
});
```

- [ ] **No transmitir información solo con el color** (forma, icono o texto de apoyo); valorar un modo para daltonismo. agnóstico
  - Por qué: en juegos de color o de tesoros por tipo, alrededor del 8 % de los hombres tiene deficiencia de color.
  - Verificar: simulación de daltonismo en las opciones de desarrollador.
  - Prioridad: **deseable**.

- [ ] **Externalizar todos los textos** y elegir el mecanismo de i18n: `flutter_localizations` con ARB (recomendado) o una clase de cadenas por locale para un MVP. flutter/flame
  - Por qué: traducir después sin tocar la UI; los textos sueltos en widgets complican el cambio.
  - Verificar: `rg "Text\('" lib` no da resultados fuera de la capa de cadenas; `l10n.yaml` y `lib/l10n/*.arb`.
  - Prioridad: **importante**.
  - Fuente: https://docs.flutter.dev/ui/accessibility-and-internationalization/internationalization

- [ ] **Declarar el idioma por app** (`localeConfig` o `supportedLocales`) y probar textos largos (alemán, español). agnóstico
  - Por qué: Android 13+ permite elegir el idioma por app; los textos largos rompen botones.
  - Verificar: Settings › Apps › Idioma; capturas en cada idioma.
  - Prioridad: **deseable**.

- [ ] **Limitar la longitud de línea en textos largos** (45 a 75 caracteres) en historia o ayudas. agnóstico
  - Por qué: Line_Length.
  - Verificar: revisión visual en tablet.
  - Prioridad: **deseable**.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

Fase 8 Errores, crashes y símbolos

- [ ] **Instalar los manejadores globales de errores** (`FlutterError.onError` y `PlatformDispatcher.instance.onError`) con registro local; sin red si la política dice "offline". flutter/flame
  - Por qué: las excepciones de Dart no tumban la app y pasan desapercibidas.
  - Verificar: `rg "FlutterError.onError|PlatformDispatcher.instance.onError" lib`; forzar una excepción en debug.
  - Prioridad: importante.

```
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  FlutterError.onError = (d) { FlutterError.presentError(d); localLog(d.exceptionAsString(), d.stack);
};
  PlatformDispatcher.instance.onError = (e, s) { localLog('$e', s); return true; };
  runApp(const MyApp());
}
```

- [ ] **Si se ofusca Dart, guardar los símbolos de cada versión** (`--obfuscate --split-debug-info=symbols/<versión>`) fuera de git, y subir los símbolos nativos de depuración a Play. flutter/flame
  - Por qué: sin símbolos, las trazas de Android vitals no se pueden leer.
  - Verificar: carpeta de símbolos por versionCode; Play Console › App bundle › símbolos de depuración nativos.
  - Prioridad: importante.
  - Fuente: https://docs.flutter.dev/deployment/android

- [ ] **Decidir si se usa un SDK de crashes** (Crashlytics, Sentry) **o solo Android vitals**, y reflejarlo en la política y en Data safety. agnóstico
  - Por qué: un SDK de crashes recopila datos, y declararlo mal en Data safety es motivo de rechazo.
  - Verificar: dependencias frente a la política de privacidad frente a Data safety.
  - Prioridad: importante.

- [ ] **No escribir en el log datos personales**; `debugPrint` solo para diagnóstico; nada de `print` en producción. agnóstico
  - Por qué: Sensitive_Data_Logging; las reglas de lint lo detectan.
  - Verificar: `rg "print\(" lib`; `avoid_print` en `analysis_options.yaml`.
  - Prioridad: importante.

- [ ] **Revisar el informe de prelanzamiento y Android vitals** (crashes, ANR, frames lentos) antes de pasar a producción. agnóstico
  - Por qué: Stability_ANR.
  - Verificar: Play Console › Pruebas › Informe de prelanzamiento; Calidad › Android vitals.
  - Prioridad: importante.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

Fase 9 Pruebas y CI

- [ ] **Mantener `flutter analyze` sin avisos** (con `flutter_lints` o más estricto). flutter/flame
  - Por qué: atrapa errores baratos antes que el dispositivo.
  - Verificar: `flutter analyze` termina sin incidencias.
  - Prioridad: importante.

- [ ] **Escribir tests unitarios de la lógica pura** (reglas, puntuación, generación procedural con semilla, validador de niveles) que llamen al código entregado sin reimplementar fórmulas. agnóstico
  - Por qué: protege el balance y las reglas frente a regresiones.
  - Verificar: `flutter test`; matriz requisito → test en la spec.
  - Prioridad: importante.

- [ ] **Probar que todo nivel es resoluble** con un juego automático sin interfaz o un validador. agnóstico
  - Por qué: evita publicar niveles imposibles.
  - Verificar: un test de validación de niveles que imprima un informe.
  - Prioridad: importante.

- [ ] **Escribir tests de widget de layout y flujo** (varios tamaños, letra a 1,3 y 2,0, insets simulados, navegación y retroceso). flutter/flame
  - Por qué: fija las garantías de las fases 2 y 5.
  - Verificar: `tester.view.physicalSize = ...`, `MediaQueryData(textScaler: ...)` y `handlePopRoute()`.
  - Prioridad: importante.

- [ ] **Si hay shaders, añadir un test de paridad** entre el modelo en Dart y el shader (tolerancia definida). flutter/flame
  - Por qué: la lógica no debe leer píxeles de la GPU; el modelo es la fuente de verdad.
  - Verificar: un test que cargue `FragmentProgram` sin interfaz y compare.
  - Prioridad: deseable.

- [ ] **Generar capturas E2E automáticas** (Playwright sobre el build web de release, o integration_test en Android). flutter/flame
  - Por qué: detecta regresiones visuales y produce capturas de partida.
  - Verificar: `npm test` en `e2e/` dentro de `verify.sh`.
  - Prioridad: deseable.

- [ ] **Añadir un CI en cada PR** que ejecute `verify.sh` (o al menos analyze y tests); la publicación del sitio va en otro workflow. agnóstico
  - Por qué: un workflow solo de GitHub Pages no protege el código.
  - Verificar: `.github/workflows/ci.yml` en verde en el PR; `main` protegido.
  - Prioridad: importante.

```
name: ci
on: [pull_request, push]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with: { channel: stable }
      - run: ./verify.sh --no-apk
```

- [ ] **Probar en dispositivos y emuladores representativos**: un físico de gama media, el último Android, un plegable, una tablet y un emulador de 16 KB. agnóstico
  - Por qué: es el entorno de prueba que pide la guía de calidad.
  - Verificar: matriz anotada en el guion de aceptación.
  - Prioridad: importante.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

Fase 10 Firma y release

- [ ] **Crear la upload key** (RSA de 2048 bits o más, alias `upload`) y guardarla **fuera de git y de la máquina compartida**, con copia de seguridad cifrada. agnóstico
  - Por qué: perder la clave obliga a pedir un reset; si se filtra, alguien puede firmar en tu nombre.
  - Verificar: `git check-ignore android/key.properties *.jks`; copia de seguridad comprobada.
  - Prioridad: crítico.
  - Fuente: https://developer.android.com/studio/publish/app-signing

```
keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA \
        -keysize 2048 -validity 10000 -alias upload
```

- [ ] **Configurar la firma de release con `key.properties`**, tolerante a que falte (para CI y builds de prueba) y **sin la clave debug en release**. flutter/flame
  - Por qué: `.toString()` sobre un valor nulo produce `"null"` y rompe el build sin clave; firmar el release con la clave debug no se puede publicar.
  - Verificar: `android/app/build.gradle.kts`; `apksigner verify --print-certs` sobre el APK generado desde el AAB.
  - Prioridad: crítico.
  - Fuente: https://docs.flutter.dev/deployment/android

```
import java.util.Properties
import java.io.FileInputStream

val keystoreProperties = Properties()
val keystorePropertiesFile = rootProject.file("key.properties")
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(FileInputStream(keystorePropertiesFile))
}
android {
    signingConfigs {
        create("release") {
            keyAlias = keystoreProperties.getProperty("keyAlias")
            keyPassword = keystoreProperties.getProperty("keyPassword")
            storeFile = keystoreProperties.getProperty("storeFile")?.let { file(it) }
            storePassword = keystoreProperties.getProperty("storePassword")
        }
    }
    buildTypes {
        release { signingConfig = signingConfigs.getByName("release") }
    }
}
```

```
# android/.gitignore
key.properties
**/*.keystore
**/*.jks
```

- [ ] **Activar Play App Signing** (obligatorio para apps nuevas con AAB). agnóstico
  - Por qué: Google custodia la clave de la app; si se pierde la upload key, se puede resetear. La futura firma híbrida PQC (Android 17) la ofrecerá Play.
  - Verificar: Play Console › Configuración › Integridad de la app › Firma de apps.
  - Prioridad: crítico.
  - Fuente: https://developer.android.com/studio/publish/app-signing

- [ ] **Publicar siempre AAB** (`flutter build appbundle --release`); los APK, solo para pruebas fuera de Play (`--split-per-abi`). agnóstico
  - Por qué: el AAB es obligatorio en Play para apps nuevas.
  - Verificar: `build/app/outputs/bundle/release/app-release.aab` ; probarlo con `bundletool` o en prueba interna.
  - Prioridad: crítico .
  - Fuente: https://docs.flutter.dev/deployment/android

- [ ] **Verificar el AAB final**: target y min SDK, permisos, versionCode, 16 KB y ausencia de librerías de depuración. agnóstico ⚠ revisar vigencia
  - Por qué: Production_Build_Quality; último control antes de subir.
  - Verificar: manifiesto fusionado de release; `readelf -lW` sobre los `.so` ; Play Console › Explorador de app bundles.
  - Prioridad: crítico .

- [ ] **Escribir las notas de versión por idioma** en el repo (`release_notes/<versión>-<idioma>.txt`). agnóstico
  - Por qué: trazabilidad y copiar y pegar directo en la consola.
  - Verificar: un archivo por versión e idioma.
  - Prioridad: deseable .

Fase 11 Publicación en Google Play y políticas

- [ ] **Publicar la política de privacidad en una URL estable** (por ejemplo `https://<dominio>/privacy.html` con GitHub Pages), bilingüe y coherente con permisos, SDK y almacenamiento local. agnóstico
  - Por qué: Play la exige; una política que no coincide con el comportamiento real se rechaza.
  - Verificar: URL accesible; comparar permisos y dependencias con el texto.
  - Prioridad: crítico.

- [ ] **Rellenar Data safety** según lo que el juego hace de verdad (aunque no recopile nada). agnóstico
  - Por qué: es obligatorio para toda app publicada.
  - Verificar: Play Console › Contenido de la app › Seguridad de los datos.
  - Prioridad: crítico.
  - Fuente: https://support.google.com/googleplay/android-developer/answer/10787469

- [ ] **Completar el cuestionario de clasificación de contenido** (IARC) y repetirlo si cambia el contenido. agnóstico
  - Por qué: sin clasificación no se puede publicar.
  - Verificar: Play Console › Contenido de la app › Clasificación de contenido.
  - Prioridad: crítico.
  - Fuente: https://support.google.com/googleplay/android-developer/answer/9859655

- [ ] **Declarar el público objetivo, los anuncios y el acceso**; si hay menores entre el público, revisar Familias y las políticas de menores. agnóstico ⚠ revisar vigencia
  - Por qué: son declaraciones obligatorias; las políticas de menores se endurecieron en 2026.
  - Verificar: Play Console › Contenido de la app (todas las secciones en verde).
  - Prioridad: crítico.

- [ ] **Preparar la ficha**: título, descripción corta y larga (por idioma), icono de 512 px, gráfico destacado de 1024×500 (sin texto pequeño ni capturas), capturas de la orientación real del juego y vídeo opcional. agnóstico
  - Por qué: Play_Feature_Graphic y Play_Misleading_Content.
  - Verificar: Play Console › Ficha de Play Store principal; capturas reducidas siguen legibles.
  - Prioridad: crítico.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Elegir la categoría «Juego» y su subcategoría** en Play (además de `appCategory` en el manifiesto). agnóstico
  - Por qué: la descubribilidad, y Play puede gestionar el flag de juego con AAB.
  - Verificar: Play Console › Configuración de la tienda.
  - Prioridad: importante.

- [ ] **Si el juego es de pago o tiene IAP**, configurar el perfil de pagos, los precios por país y los impuestos; para IAP, usar Play Billing. agnóstico
  - Por qué: sin perfil de pagos no se pueden poner precios; los IAP fuera de Play Billing infringen la política.
  - Verificar: Play Console › Configuración › Perfil de pagos; Monetizar › Productos.
  - Prioridad: crítico (si aplica).

- [ ] **Pasar por prueba interna y, si la cuenta lo exige, cerrada** (las cuentas personales nuevas necesitan testers durante varios días antes de producción) antes de producción. agnóstico ⚠ revisar vigencia
  - Por qué: el informe de prelanzamiento y el feedback real llegan antes que a todos los usuarios.
  - Verificar: Play Console › Pruebas › Prueba interna o cerrada.
  - Prioridad: **importante**.

- [ ] **Confirmar la verificación de desarrollador Android** (identidad y registro de paquetes). agnóstico ⚠ revisar vigencia
  - Por qué: ya se exige en algunos países; el despliegue global está previsto para 2027.
  - Verificar: Play Console › Configuración › Cuenta de desarrollador.
  - Prioridad: **importante**.
  - Fuente: https://developer.android.com/developer-verification

- [ ] **Publicar un sitio promocional mínimo** (página, vídeo, enlace a Play y política) desplegado por CI. agnóstico
  - Por qué: sirve de URL de privacidad y de destino de marketing.
  - Verificar: workflow de Pages en verde; dominio propio con HTTPS.
  - Prioridad: **deseable**.

Fase 12 Mantenimiento post-lanzamiento

- [ ] **Revisar cada año el requisito de API objetivo** (Play suele subirlo en agosto, con prórroga hasta noviembre) y actualizar Flutter, AGP y NDK. agnóstico ⚠ revisar vigencia
  - Por qué: por debajo de lo exigido no se pueden publicar actualizaciones, y las apps existentes dejan de ser visibles para usuarios nuevos.
  - Verificar: https://support.google.com/googleplay/android-developer/answer/11926878; Play Console › Panel (avisos de políticas).
  - Prioridad: crítico.

- [ ] **Leer los cambios de comportamiento de cada Android nuevo** (edge-to-edge, orientación, retroceso, 16 KB…) y probar con las opciones de compatibilidad. agnóstico
  - Por qué: Android_Platform_Compatibility.
  - Verificar: emulador con la última API; Developer options › App Compatibility Changes.
  - Prioridad: importante.
  - Fuente: https://developer.android.com/about/versions/16/behavior-changes-16

- [ ] **Mantener las dependencias al día** (`flutter pub outdated`) y priorizar las que traen código nativo. flutter/flame
  - Por qué: SDK_Maintenance; los plugins nativos son la fuente típica de problemas de 16 KB y de crashes.
  - Verificar: `flutter pub outdated` ; changelog de cada plugin nativo.
  - Prioridad: importante.
  - Fuente: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Vigilar Android vitals y las reseñas** y responder a los fallos que se reproduzcan. agnóstico
  - Por qué: Play_User_Reviews y Stability_ANR; los umbrales de vitals afectan a la visibilidad.
  - Verificar: Play Console › Calidad › Android vitals, y Valoraciones y reseñas.
  - Prioridad: importante.

- [ ] **Tener la documentación sincronizada con la versión publicada** (roadmap marcado, README sin versiones contradictorias, estado actualizado). agnóstico
  - Por qué: la documentación obsoleta desorienta a los agentes y al propio autor meses después.
  - Verificar: la versión del README, el plan y `pubspec.yaml` coinciden; roadmap de la fase de release cerrado.
  - Prioridad: deseable.

- [ ] **Volcar las lecciones de cada juego en esta plantilla o skill** (qué se resolvió, qué faltó). agnóstico
  - Por qué: mejora continua entre proyectos.
  - Verificar: entrada en el changelog de la skill.
  - Prioridad: deseable.

Notas para convertir en skill

Nombre sugerido: `creacion-juegos-android`

Cuándo usarla (trigger)

- El usuario quiere crear, preparar o auditar un juego móvil para Android ("nuevo juego", "publicar en Play", "checklist de release", "revisar mi juego antes de subirlo").
- Hay un repo Flutter con `flame` en `pubspec.yaml`, o un proyecto Android cuyo propósito principal es un juego.
- No usarla para apps no lúdicas: en ese caso no aplican `appCategory="game"` ni sus excepciones de orientación.

Entradas que debe pedir la skill

1. Modo: **nuevo proyecto** (fases 0 a 12) o **auditoría** de un repo existente (fases 1 a 11 contra el código).
2. Nombre del juego y `applicationId` (dominio inverso), y si ya está publicado en Play.
3. Orientación (vertical u horizontal) y si debe admitir tablet o plegable.
4. Stack: Flutter + Flame (por defecto) u otro (se aplican solo los ítems `agnóstico`).
5. Monetización: gratis, pago único, IAP o anuncios (cambia Data safety, Billing y familias).
6. Datos y red: offline o con red, analítica, SDK de crashes (cambia la política y Data safety).
7. Idiomas de la UI y de la ficha.
8. Público objetivo (¿incluye menores?).
9. Nivel de autonomía del agente y reglas de parada (qué decide siempre el dueño).

Pasos que se pueden automatizar (script `verify.sh` o `audit.sh`)

Check	Cómo
Analyze y tests	`flutter analyze` · `flutter test`
SDK efectivos	`flutter build appbundle` y leer `<uses-sdk>` del manifiesto fusionado de release
16 KB	descomprimir el AAB y `readelf -lW base/lib/*/*.so | awk '/LOAD/{print $NF; exit}'` ≥ `0x4000`
Flags del manifiesto	`grep` de `appCategory="game"`, `enableOnBackInvokedCallback="true"`, `exported`, `uses-permission`
Edge-to-edge	`rg "immersiveSticky|windowOptOutEdgeToEdge"` (aviso si aparece con target ≥ 36)
Icono por defecto	`md5sum mipmap-*/ic_launcher.png` frente a la plantilla de Flutter
Splash claro	`grep "Theme.Light" res/values/styles.xml` en un juego oscuro (aviso)
Firma	`signingConfigs.getByName("debug")` en el release (error); `key.properties` y `*.jks` en `.gitignore`
Versión	`version:` de `pubspec.yaml` frente a README y plan; versionCode mayor que el último subido (si se conoce)
Lifecycle	`rg "lifecycleStateChange|WidgetsBindingObserver|pauseWhenBackgrounded"`; si se sobrescribe, comprobar `super.`
Retroceso	`rg "PopScope" lib/**/game*`
A11y e i18n	`rg "Semantics"`; `rg "Text\('"` fuera de la capa de cadenas; tests `meetsGuideline`
Errores	`rg "FlutterError.onError|PlatformDispatcher.instance.onError"`; `rg "print\("`
Permisos y política	lista de `uses-permission` y dependencias con red frente al texto de `privacy.html`
CI	existe `.github/workflows/*` que ejecuta analyze y test (no solo Pages)

Pasos que siguen siendo humanos o de Play Console (la skill solo los recuerda): Play App Signing, Data safety, clasificación IARC, público objetivo, ficha y gráficos, perfil de pagos, pruebas internas o cerradas, verificación de desarrollador, sensación de juego y dificultad (guion de aceptación), pruebas en dispositivo físico.

Mantenimiento de la skill: cada ítem marcado `⚠ revisar vigencia` debe contrastarse con su fuente al ejecutar la skill. Conviene anotar en la skill la fecha de la última revisión (esta versión: 19-oct-2026).